

Final Project

Course & Student Platform

Django MVT • Django REST Framework • Token Authentication • RAG • Multi-Agent AI

Build a platform where teachers create courses, students enroll in them, and admins moderate everything. The project combines Django MVT for the web interface, Django REST Framework for the API, and an agentic AI layer with RAG-powered course recommendations and a multi-agent teaching assistant — putting everything from all sessions into one real-world application.

■ Project Overview — The Three Roles

ADMIN	TEACHER	STUDENT
Manages the platform. Approves courses before they go live. Can deactivate any user account.	Creates and manages courses. Courses start unpublished and need admin approval. Can only edit their own courses.	Browses published courses and enrolls in them. Can be enrolled in multiple courses simultaneously.

■ Models

Profile — extends the built-in User

Users choose their role — **student** or **teacher** — at registration. No approval is required; the selected role is assigned immediately.

Field	Type	Notes
user	ForeignKey to User (unique=True)	Links to Django's built-in User model
role	CharField with choices	Choices: student / teacher / admin. Selected by user at registration.
bio	TextField	Optional — blank=True, null=True

Category — groups courses by topic

Field	Type	Notes
name	CharField(max_length=100)	e.g. Programming, Design, Business
slug	SlugField(unique=True)	URL-friendly name. e.g. 'web-development'

Course — the main model

Note: *category is a ForeignKey on Course linking it to a Category. Every course must belong to a category; it is required.*

Field	Type	Notes
title	CharField(max_length=200)	
description	TextField	
teacher	ForeignKey to User	The teacher who created this course
category	ForeignKey to Category	Required — every course must belong to a category
level	CharField with choices	Choices: beginner / intermediate / advanced
is_published	BooleanField	Default False. Admin sets True to approve and publish.
created_at	DateTimeField	auto_now_add=True
updated_at	DateTimeField	auto_now=True

Enrollment — links students to courses

Field	Type	Notes
student	ForeignKey to User	The enrolled student
course	ForeignKey to Course	The course being enrolled in
enrolled_at	DateTimeField	auto_now_add=True

■ ■ Add `unique_together = ('student', 'course')` in class `Meta` on the `Enrollment` model. This prevents a student from enrolling in the same course twice at the database level.

CourseDocument — AI knowledge base per course

Stores uploaded materials (PDFs, text files) attached to a course. These documents are chunked and indexed into the vector store to power the RAG teaching assistant.

Field	Type	Notes
course	ForeignKey to Course	The course this document belongs to
title	CharField(max_length=200)	Display name for the document
file	FileField	Uploaded file (PDF or .txt)
uploaded_at	DateTimeField	auto_now_add=True

■ ■ Only the course owner (teacher) can upload documents. Documents are indexed automatically on save using a Django `post_save` signal.

■ Permission Rules

Student Rules

Action	Allowed?	Condition
Browse published courses	✓ Yes	Guests can browse too — no login required
Enroll in a course	✓ Yes	Must be logged in. Account must be active.
Enroll twice in the same course	✗ No	Return a clear error message — not a 500 crash
See own enrollments	✓ Yes	Only their own. Never another student's.
Create or edit courses	✗ No	Students have no access to course management
Use course AI assistant	✓ Yes	Must be enrolled in the course.
Use course recommender	✓ Yes	Must be logged in.
Access if deactivated	✗ No	is_active=False blocks login entirely

Teacher Rules

Action	Allowed?	Condition
Create a course	✓ Yes	Account must be active. Course starts as is_published=False.
Edit own course	✓ Yes	Only their own courses. Account must be active.
Delete own course	✓ Yes	Only their own courses. Account must be active.
Edit another teacher's course	✗ No	Return 403.
See own unpublished courses	✓ Yes	Teachers see all their courses including unpublished ones.
Upload course documents	✓ Yes	Only for their own courses. Used to feed the RAG assistant.
Enroll in courses as a student	✗ No	Teachers cannot enroll.
Access if deactivated	✗ No	is_active=False blocks login entirely

Admin Rules

Action	Allowed?	Condition
Approve a course	✓ Yes	Set is_published=True. Done via Django Admin.
Deactivate a user	✓ Yes	Set is_active=False on the User. Done via Django Admin.
Reactivate a user	✓ Yes	Set is_active=True. All their data is immediately accessible again.
Delete user data	✗ No	Deactivation only. Data is never deleted.

■■ is_active=False blocks login entirely — the user cannot access the platform at all. Their courses and enrollments remain fully intact. Reactivating restores full access without any data loss.

■ Django MVT Pages

URL	Page	Who can access
/	Course list — published only. Search by title/description. Filter by category and level.	Everyone incl. guests
/courses/<pk>/	Course detail. Shows Enroll button for active students.	Everyone incl. guests
/courses/create/	Create course form.	Active teachers only
/courses/<pk>/edit/	Edit course — pre-filled with existing data.	Owner teacher, active
/courses/<pk>/delete/	Delete confirmation page.	Owner teacher, active
/courses/<pk>/enroll/	POST action — no page, redirects after.	Active students only
/courses/<pk>/documents/	Upload course documents for RAG indexing.	Owner teacher, active
/courses/<pk>/assistant/	Chat UI — multi-agent teaching assistant for this course.	Enrolled students only
/my-enrollments/	Student's enrolled courses.	Students only
/my-courses/	Teacher's courses — published and unpublished.	Teachers only
/register/	Registration — user selects student or teacher role at signup.	Public
/login/	Login page.	Public
/admin/	Django Admin — approve courses, manage users.	Admins only

Search and Filtering on the Course List

Query param	Example	Behavior
?search=	?search=python	Filter courses where title or description contains the term
?category=	?category=programming	Filter by category slug
?level=	?level=beginner	Filter by level — beginner / intermediate / advanced

■ REST API Endpoints

All endpoints use Token Authentication. Protected endpoints require the `Authorization: Token` header.

Authentication

Method	Endpoint	Description	Auth
POST	/api/v1/auth/register/	Register. Body: {username, password, role}. Returns token.	No
POST	/api/v1/auth/token/	Get token. Body: {username, password}.	No

Courses

Method	Endpoint	Description	Auth
GET	/api/v1/courses/	List published courses. Supports ?search=, ?category=, ?level=	No
POST	/api/v1/courses/	Create course. Teacher assigned from request.user.	Teacher
GET	/api/v1/courses/<pk>/	Course detail.	No
PUT	/api/v1/courses/<pk>/	Update course. Owner teacher only.	Teacher
DELETE	/api/v1/courses/<pk>/	Delete course. Owner teacher only.	Teacher

Enrollments

Method	Endpoint	Description	Auth
POST	/api/v1/courses/<pk>/enroll/	Enroll the requesting student in this course.	Student
GET	/api/v1/my-enrollments/	List the student's enrolled courses.	Student
GET	/api/v1/my-courses/	List the teacher's created courses.	Teacher

■ ■ The enroll endpoint is a custom `@action` on the `CourseViewSet` — not a separate view. Use `@action(detail=True, methods=['post'])`.

Agentic AI Endpoints

Method	Endpoint	Description	Auth
POST	/api/v1/courses/recommend/	RAG recommendation. Body: {query}. Returns ranked published courses based on query + student enrollment history.	Student
POST	/api/v1/courses/<pk>/chat/	Multi-agent teaching assistant. Body: {message}. Returns AI response grounded in course documents.	Student (enrolled)
POST	/api/v1/courses/<pk>/documents/	Upload a document to the course knowledge base. Triggers RAG indexing.	Teacher (owner)

■ ■ The *recommend* and *chat* endpoints are custom *@actions* on the *CourseViewSet*. *chat* requires the student to be enrolled — check before invoking the agent.

■ Agentic AI Architecture

Overview

The platform includes two AI-powered features built on top of the Django backend: a RAG-based course recommender and a multi-agent teaching assistant per course. Both are exposed via DRF API endpoints and rendered in the MVT templates.

Feature 1 — RAG Course Recommender

When a student sends a natural language query (e.g. "I want to learn machine learning from scratch"), the recommender retrieves the most semantically relevant published courses using vector similarity search over the course catalog.

- All published course titles, descriptions, and categories are embedded and stored in a vector store (e.g. ChromaDB or FAISS).
- On query, the student's message is embedded and the top-N most similar courses are retrieved.
- The student's existing enrollment history is used to filter out already-enrolled courses.
- Results are returned ranked by relevance with a short AI-generated explanation for each recommendation.
- The vector store is updated whenever a course is published or its content changes (use Django signals).

Feature 2 — Multi-Agent Teaching Assistant

Each published course has a dedicated AI assistant that enrolled students can chat with. The system uses an orchestrator-agent pattern to route questions to the right sub-agent.

Agent	Role	Behaviour
Orchestrator	Router	Receives the student's message and decides which sub-agent should handle it. Passes context (course info, chat history) to the chosen agent.
RAG Agent	Grounded Q&A;	Answers questions using course documents as the knowledge base. Retrieves relevant chunks via vector search, then generates a grounded answer. Returns source references.
General Agent	Broad support	Handles questions related to the course subject that fall outside the uploaded documents. Uses the LLM's general knowledge, clearly noting when the answer is not from course materials.

Agent Flow

- Student sends a message to `/api/v1/courses//chat/` with their question.
- The Orchestrator agent receives the message plus the course context (title, description, category) and the last N chat turns.
- The Orchestrator classifies the question: document-grounded (→ RAG Agent) or general subject help (→ General Agent).
- The selected sub-agent generates a response and returns it with a source tag (RAG: document chunks used; General: 'general knowledge').
- The response is stored in the session/DB and returned to the student. The chat history is maintained per student per course.

Technical Stack for AI Layer

Component	Recommended Tool / Approach
LLM	OpenAI GPT-4o or any OpenAI-compatible API (e.g. Anthropic Claude via API)
Embeddings	OpenAI text-embedding-3-small or sentence-transformers
Vector store	ChromaDB (local, easy setup) or FAISS for lightweight deployment
Agent framework	LangChain (recommended) or plain Python with OpenAI function calling
RAG chunking	Split documents into ~500-token chunks with 50-token overlap
Signal hook	Django post_save on CourseDocument to trigger re-indexing automatically
Chat history	Store per-student per-course in DB as a JSON field or a ChatMessage model

■■ The LLM API key must be stored in an environment variable — never hardcoded. Add `OPENAI_API_KEY` (or equivalent) to your `.env` file and load it with `python-decouple`.

■ Technical Requirements

Django — MVT

- Function-based views for all web pages
- ModelForm for course creation and editing
- `commit=False` to assign teacher on creation. `instance=` to pre-fill the edit form.
- `@login_required` on all protected views
- Role-based access checks inside views — redirect or 403 if the role doesn't match
- Template inheritance — one `base.html` that all pages extend
- Django Admin with `list_display`, `list_filter`, `search_fields` on all models
- Custom admin action to bulk-approve multiple courses at once

Django REST Framework

- `ModelSerializer` for Course and Enrollment
- `ModelViewSet` for courses with `get_queryset` override for filtering and ownership
- `perform_create` to assign teacher = `request.user` automatically
- Custom permission classes — `IsTeacher`, `IsStudent`, `IsCourseOwner`
- Token Authentication on all protected endpoints
- `@action` for enroll, recommend, chat, and document upload endpoints
- Return meaningful error messages — not just status codes

Agentic AI

- RAG pipeline: embed all published course data into a vector store on publish/update
- Recommendation `@action`: accepts a natural language query, returns ranked courses with AI explanation
- Orchestrator agent: routes student questions to RAG Agent or General Agent based on intent
- RAG Agent: retrieves relevant document chunks and generates a grounded answer with source references
- General Agent: handles broader subject questions, clearly labels responses as general knowledge
- Chat history: persist per-student per-course conversation turns in the database
- CourseDocument model: teachers upload materials; `post_save` signal triggers re-indexing
- LLM API key loaded from environment variable — never hardcoded

General

- `requirements.txt` with all dependencies pinned
- Migrations committed to the repository
- At least 3 categories seeded in the database
- README with setup instructions including how to configure the LLM API key and vector store

■ ■ Never delete user data when deactivating. *is_active=False* is the only change. Courses and enrollments must remain fully intact.

■ Bonus Challenges

Optional. Only attempt after all core requirements are fully working and tested.

Bonus 1 — Soft Ban (`is_banned`)

Add `is_banned = BooleanField` to the Profile model. Unlike `is_active` which blocks login, a banned user can still log in and see their data but cannot perform actions.

Role	What <code>is_banned</code> blocks
Student	Cannot enroll in new courses or use the AI assistant. Can still see existing enrollments.
Teacher	Cannot create, edit, or delete courses. Can still see existing courses.

Implement as a custom `IsNotBanned` permission class. Check `request.user.profile.is_banned` and raise `PermissionDenied` if `True`.

Bonus 2 — Rating Model

Students can rate a course they are enrolled in (score 1–5).

Field	Type	Notes
student	ForeignKey to User	The student giving the rating
course	ForeignKey to Course	The course being rated
score	IntegerField	Value 1–5. Validate — reject anything outside range.
created_at	DateTimeField	<code>auto_now_add=True</code>

- ★ A student can only rate a course they are enrolled in — check before saving
- ★ A student can only rate the same course once — use `unique_together`
- ★ Show average rating on the course detail page using `Avg` from `django.db.models`
- ★ Add average rating to the `CourseSerializer` using `SerializerMethodField`

Bonus 3 — Unenroll

Add a custom `@action` on the `CourseViewSet`. Students can leave a course they are enrolled in.

- ★ Endpoint: `POST /api/v1/courses/{id}/unenroll/`
- ★ Only the enrolled student themselves can unenroll
- ★ If not enrolled, return a clear error — not a 500
- ★ Add an Unenroll button on the course detail web page too

■ What Will Be Reviewed

Area	What is being checked
Permission logic	Can a deactivated student enroll? Can a teacher edit another teacher's course? These are checked first.
Unpublished courses	Unpublished courses must never appear on the public list or be accessible by non-teachers.
Double enrollment	Enrolling twice must return a clear error — not crash with a 500 IntegrityError.
Owner assignment	The teacher field must always be set from request.user — never from user input.
Deactivation behavior	is_active=False blocks login but all data stays intact. Reactivation restores full access.
API authentication	Protected endpoints return 401 without a token and 403 with wrong permissions.
RAG recommender	Returns semantically relevant courses for a query. Excludes already-enrolled courses. Vector store updates on publish.
Multi-agent assistant	Orchestrator correctly routes to RAG vs General agent. RAG responses cite source chunks. Chat history is maintained per student per course.
Agent access control	Only enrolled students can use the chat endpoint. Only the course owner can upload documents.
Code quality	Permission logic in permission classes — not repeated across every view. Agent logic in dedicated service modules.

■ Suggested Build Order

Build in this sequence to avoid blockers. Get the full Django stack working before adding the AI layer.

#	Step	Details
1	Project setup	Create project and app. Install DRF and AI dependencies. Configure settings.
2	Models	Build Profile, Category, Course, Enrollment, CourseDocument. Run migrations.
3	Django Admin	Register all models. Create superuser. Seed categories.
4	Registration & Login	Register with role selection (student or teacher) at signup, login, logout.
5	Course web pages	Course list with search/filter, detail, create, edit, delete.
6	Enrollment web pages	Enroll action, my-enrollments, my-courses pages.
7	DRF serializers	CourseSerializer and EnrollmentSerializer.
8	DRF views & URLs	CourseViewSet with filtering. Enroll @action. Enrollment endpoints.
9	Permission classes	IsTeacher, IsStudent, IsCourseOwner.
10	Token Authentication	Register API, token endpoint. Test the full flow with Postman.
11	RAG recommender	Set up vector store. Embed published courses. Build recommend @action. Test with natural language queries.
12	Multi-agent assistant	Build Orchestrator, RAG Agent, General Agent. Wire to chat @action. Add chat UI page. Test routing and history.
13	Bonus (optional)	is_banned, Rating, Unenroll — only after core + AI features are complete.

■■ Build the full Django + DRF stack first (steps 1–10). Your models, permissions, and API will be solid by the time you add the agentic layer on top (steps 11–12).

Final Project • Course & Student Platform • Django + DRF + RAG + Multi-Agent AI